```
When he submits his application online
Then his application should be created in a pending state
And he should be sent a PDF contract to sign by email
```

The expected outcomes. ❸

A higher-level view of the submission process. ❷

This version doesn't mention the user interface at all and focuses primarily on the submission process ❷ and the expected outcomes ❸. You summarize the fields you need to enter as a table ❶.

But this leads us to another question: do you really need this level of detail? For this scenario, the values entered have no importance, so you can skip them entirely and simplify the scenario even further:

```
Scenario: Register for online banking
Given that Bill wants to register for online banking
When he submits his application online
Then his application should be created in a pending state
And he should be sent a PDF contract to sign by email
```

What Bill wants

What he does to achieve it

The expected outcomes

This scenario does a much better job of focusing on the business actions that you're describing. A scenario should only contain information that contributes to communicating the core business actions described by the scenario: anything else is waste.

5.4.3 Writing expressive Then steps

The `Then` step is where the testing takes place—this is where you describe what outcome you expect.

A common anti-pattern among new BDD practitioners is to mix the `When` and `Then` steps: a well-written scenario should only test a single action. Many teams new to BDD write scenarios that look like this:

```
Scenario: Updating my account address
Given I am logged in as a bank client
Then I should see the home page
When I select the 'accounts' menu
Then I should see the 'Accounts' page
And I should see a list of accounts
When I click on 'Edit'
Then I should see 'Account Details'
When I type '100 Main Street' into the street
And I type 'Armidale' into the city
And I type 'Update'
Then I should see the updated account details on the account summary page
```

This is more of a test script than a description of a behavior, which is very much a BDD anti-pattern. Scenarios written this way communicate poorly and are hard to maintain because they're very tightly coupled to the implementation.

> **TIP** Good scenarios are *declarative*, not imperative. They describe the requirements in terms of *what* a feature should do, not *how* it should do it.

Once again, you can simplify this scenario by removing UI-specific implementation details and data that the business doesn't really care about. Doing this, you could write a much cleaner version of the scenario along the following lines: